

ABSTRACT

Este trabajo desarrolla un modelo predictivo del peso de perros en adopción a partir de sus características físicas, conductuales, y de historial de ingreso. Sobre un conjunto de 42,000 registros, se implementaron dos modelos de regresión lineal. Uno mediante descenso de gradiente construido desde 0 y el otro utilizando scikit-learn. Fueron evaluados sobre conjuntos independientes de entrenamiento, validación y prueba (60/20/20). Ambos modelos alcanzaron un R^2 de hasta el 0.88 y un error absoluto promedio de 3.12kg, presentando un sesgo bajo, varianza baja y un buen ajuste, sin mejoras observadas al aplicar regularización L2. Sin embargo, se realizó un experimento en donde se generó overfitting y se aplicó regularización L2, así mismo se generaron mejoras observadas.

1. Introducción

1.1 Contexto General

Los refugios de animales son unos de los varios sectores de hoy en día en donde manejan constantemente grandes volúmenes de información asociada a los animales bajo su cuidado, incluyendo variables demográficas y morfológicas: raza, edad, sexo, coloración, estado de salud, comportamiento, historial de ingreso y resultado del proceso de adopción. Gracias a la gran variación de los datos manejados nos abre la posibilidad de utilizar las técnicas de análisis estadístico y aprendizaje automático para transformar los registros operativos en información predictiva, la cual nos ayudará a la toma de decisiones.

De hecho, ya existen iniciativas como es ‘Shelter Animal Count’, las cuales analizan datos de refugios a nivel nacional y desde el 2024. Shelter Animal Count, ya emplea un modelo de aprendizaje automático para generar estimaciones precisas de sus datos utilizando variables del animal. Gracias a todo esto existe la posibilidad de identificar patrones y apoyar a la toma de decisiones [1].

Así mismo, ya existen otro tipo de modelos que estiman las características de un

animal a partir de datos disponibles, reduciendo la necesidad de realizar mediciones manuales. De hecho, Amir Zadeh, y colaboradores. enfocaron un análisis de estudio en donde implementaron redes neuronales, y árboles de decisión junto con otros tipos de técnicas de minería de datos, todo esto para predecir la velocidad de adopción. Esto fue gracias a la relación que existía entre la edad, raza, cantidades de publicación y el estado de esterilización, por lo que se dieron cuenta que dichas variables eran los factores que contribuyen más a la adopción de una mascota [2].

El trabajo de este documento, es similar al análisis proporcionado por el equipo de Amir y colaboradores. En donde el enfoque principal es el peso del perro y ver la correlación que existe con los otros factores.

1.2 Descripción del dataset

En este caso se está utilizando el dataset de “Dog_Adoption_Master.csv”, el cual fue proporcionado por la plataforma Kaggle. En este caso el dataset contiene alrededor de 42,000 instancias de perros que pasaron por el proceso de adopción dentro de un refugio, descritos por 32 atributos. Los atributos pueden ser agrupados en tres tipos: (1) Características físicas del perro (edad,

peso, tamaño, grupo de raza, tipo de ingreso al refugio, necesidades médicas. (2) Rasgos de comportamiento (nivel de agresividad, ansiedad por separación, reactividad hacia otros perros, nivel de energía, nivel de entrenamiento.). Cabe mencionar, que las agrupaciones de nivel 2 (Rasgos de comportamiento, son medidas del 0 - 10. Indicando que el 0 es el nivel más bajo de la categoría y 10 el más alto. (3) Información sobre el proceso de adopción y el hogar adoptante. (Tipo de vivienda, presencia de niños o mascotas, número de visitas antes de la adopción y si el perro fue devuelto al refugio.). En la siguiente tabla se puede observar la información detallada de cada uno de los features incluyendo una descripción corta y el tipo de dato que es.

Variable	Tipo	Descripción
Adoption_id	Texto	Identificador único del registro de adopción
Age_years	Numérica	Edad del perro, en años.
Size	Categórica	Categoría de tamaño (Small, medium, large, xlarge)
Weight_in_kg	Numérica	Peso del perro, en kg
Breed_group	Categórica	Grupo de raza (Hound, terrier, herding, mixed, sporting, toy, working)
Intake_type	Categórica	Forma de ingreso al refugio (stary, Owner_surrender, transfer, born_in_care)
Days_in_shelter	Numérica	Días que el perro permaneció en el refugio antes de ser adoptado.
Previously_returned	Categórica	Nivel de necesidades

		médicas (None, Minor, Chronic)
Neutered	Binaria	Si el perro está esterilizado.
Aggression_score	Numérica	Agresividad, escala 0-10.
Anxiety_separation	Numérica	Ansiedad por separación, escala 0-10.
Reactivity_to_dogs	Numérica	Reactividad hacia otros perros, escala 0-10
Energy_level	Numérica	Nivel de energía, escala 0-10
Training_level	Numérica	Nivel de obediencia, escala 0-10
House_trained	Binaria	Si el perro está entrenado para estar dentro de casa
First_time_owner	Binaria	Si el adoptante es dueño de perros por primera vez
Household_has_kids	Binaria	Si hay niños en el hogar del adoptante.
Household_has_pets	Binaria	Si hay otras mascotas en el hogar del adoptante.
Home_type	Categórica	Tipo de vivienda del adoptante.
Has_yard	Binaria	Si el hogar del adoptante tiene patio.
Hours_alone_per_day	Numérica	Horas promedio que el perro estaría solo al día.
Adopter_activity_level	Numérica	Nivel de actividad física del adoptante
Visits_before_adoption	Numérica	Número de visitas al refugio antes de adoptar.

Met_resident_pets	Binaria	Si el perro convivió con las mascotas del adoptante.
Adoption_counseling	Binaria	Si el adoptante recibió asesoría antes de adoptar
Expectation_score	Numérica	Qué tan alineadas estaban las expectativas del adoptante.
Energy_mismatch	Numérica	Diferencia entre la energía del perro y la del adoptante.
Size_home_mismatch	Binaria	Posible desajuste entre tamaño del perro y tipo de vivienda
Return_reason	Categórica	Motivo de devolución, si aplica
Days_to_return	Numérica	Días hasta la devolución, si aplica
Returned	Binaria	Si el perro fue devuelto al refugio.

Tabla 1.1: “Tabla de variables del dataframe”

1.3 ¿Qué se está prediciendo?

En este trabajo nuestra variable objetivo es “**Weight_in_kg**”, en otras palabras el peso del perro en kilogramos. A pesar de esto, el proceso para elegir nuestra variable a predecir no resultó fácil debido a que al inicio mi hipótesis fue predecir la variable de “Days_in_shelter”(los días que tarda un perro en ser adoptado), esta fue mi variable objetivo al principio debido a que es una pregunta de negocio más interesante para un refugio, debido a que es una de las variables más importantes para los refugios dado que quieren que las mascotas encuentren un hogar lo más rápido posible. Sin embargo, durante el análisis exploratorio, se mostró que ninguna de las variables físicas ni de comportamiento disponibles tenían una correlación lineal apreciable con “Days_in_shelter”, cómo se

va a apreciar posteriormente en la sección 3 (Análisis Exploratorio de Datos (EDA)). Todos las correlaciones con la variable de “Days_in_shelter” estaban por debajo de 0.05 en valor absoluto. Gracias a esto, se formó la decisión de no utilizar esta variable como mi variable objetivo debido a que este tipo de variables producen modelos que apenas aprenden, lo cual no demostraría que nuestro algoritmo funciona de la mejor manera. Como mencionan Kuhn y Johnson (2013), en “Applied Predictive Modeling” al retener variables con baja o correlaciones lineales nulas introducen ruido en lugar de señal predictiva. Por lo tanto, esto puede degradar la estabilidad del modelo y proporcionar un aprendizaje deficiente. Debido al ruido que se está generando. [3]

Por otra parte, en “**Weight_in_kg**” muestra una correlación fuerte y clara con otra variable del dataset, el tamaño del perro (“**Size**”). Sin embargo, aunque parezca una variable no tan útil para los refugios, de hecho les ayuda bastante debido a que les permite a los refugios estimar dosis de medicamentos o proporciones de comida para los perros y también ayuda a crear una validación con los datos registrados en la base de datos del refugio, esto se debe a que a la gran escala y corto tiempo que existen en los refugios puede que haya errores en algunos de los datos. Gracias a que Weight_in_kg ayuda a señalar estadística real y existe utilidad práctica es por lo que se eligió weight_in_kg, como variable objetivo final del proyecto.

1.4 ¿Regresión o clasificación?

Para empezar un problema de regresión busca predecir una variable cuantitativa, es decir un valor numérico que puede tomar cualquier número dentro de un rango continuo. Por otra parte, un problema de clasificación lo que busca es predecir una variable categórica, ya sea predecir una variable categórica que pertenece a un conjunto finito, niveles de riesgo o tipos de

clases. Es por eso, que la diferencia no está en el algoritmo que se utilice para resolver sino que está en lo que se quiere predecir, ya sea si la variable objetivo es numérica y continua, lo cual la haría de un problema de regresión, o si la variable es una categoría, lo hace un problema de clasificación. [4]

Gracias a esto podemos definir que nuestra variable de “Weight_in_kg” es una variable numérica continua (rango observado: 2.0 a 70.6 kg), es por eso que no es categórica debido a que es una variable numérica. Por lo tanto, es un problema de regresión. Si nuestra variable objetivo fuera “returned”, o sea si el perro fue devuelto al refugio 0 o 1, este sería un problema de clasificación debido a que pertenece a dos tipos de clases en vez de ser un rango numérico continuo. Dado que el objetivo de nuestro modelo es estimar cuántos kilogramos pesa un perro, es por eso que se utiliza la regresión lineal y no la regresión logística ni ningún otro tipo de clasificador.

2. Extracción, Transformación y Carga de Datos (ETL)

2.1 ¿Qué es ETL?

ETL es un acrónimo de “Extract, Transformación y Carga de datos” es un proceso mediante el cual los datos se obtienen de una o varias fuentes, se limpian y transforman a un formato más analizable. Finalmente, se cargan en un destino en donde se pueden analizar o modelar. El ETL es un marco de trabajo estándar en cuanto a la ingeniería de datos esto es debido a que ayuda a preparar la información desde antes de cualquier análisis o entrenamiento de modelos. [5]. El ETL tradicionalmente se compone en tres etapas:

Extracción: Es en donde se obtienen los datos crudos desde la fuente original, en esta área no se hace ningún tipo de cambio a la data.

Transformación: Es en donde se limpian, validan y reestructuran los datos. También, es en donde se manejan los valores nulos, la eliminación de duplicados y la codificación de variables categóricas. Todo esto es para que los datos estén listos para proseguir con el área de análisis.

Carga: Es en donde se almacena el resultado final en un formato listo para ser consumido por las siguientes etapas del proyecto.

Aunque el ETL surgió para los datos empresariales, el mismo marco de tres etapas aplica directamente a cualquier proyecto de ciencias de datos, esto se debe a que ayuda a tener un proceso en cuestión de los datos ya que, en muchos casos los datos no son consistentes, incluyen errores y tienen formatos diferentes causando inconsistencias dentro de los modelos. Es por eso la importancia de aplicar un ETL en cualquier modelo que se esté trabajando con datos, ya que ayuda a evitar muchos de estos problemas que pueden afectar la manera en la que funciona nuestro modelo. A continuación la sección 2.2 -2.6 documenta cómo se aplicó cada etapa del dataset en el proyecto.

2.2 Extracción

Dentro de la etapa de extracción, como mencionado en la sección 2.1, es en donde se obtienen los datos crudos desde la fuente original (Kaggle), esto sin aplicar ninguna transformación. En este proyecto los datos con los que estamos trabajando están en un archivo .csv llamado “dog_adoption_master.csv”, el cual contiene los registros históricos de adopciones de un refugio de perros.

Utilizando la librería de pandas se pudo extraer los datos. Gracias a pandas los tabula en un Dataframe, el cual nos ayuda a poder trabajar con la información de una manera más clara y sencilla.

Posteriormente después de la extracción se verificó que la carga fuera correcta utilizando los diferentes comandos como “df.shape, df.info() y df.head()”. Gracias a esto nos confirma que estamos trabajando con datos de 42,000 instancias y 32 atributos. No solo eso sino que, hay una mezcla de tipo de datos 12 atributos son float64, 13 son int64 y 7 son strings. Gracias a esta extracción podemos ver con qué datos estamos trabajando facilitando las siguientes etapas del proyecto.

2.2 Análisis y revisión de calidad de datos.

Gracias a la revisión descrita en la sección anterior se hicieron tres aspectos para mejorar la calidad de los datos con los que estamos trabajando.

Para empezar la existencia de los valores nulos, dentro de nuestro dataframe existía un atributo de los 32, el cual contaba con 35,703 valores nulos, debido a que estamos trabajando con 42,000 instancias esto es el 85% de las instancias con datos nulos. El atributo que contaba con esto era “Days_to_return”, este es un atributo interesante debido a que solo aplica a los perros que fueron devueltos al refugio. En otras palabras, los valores nulos eran para los perros que no aplicaban. Es por eso que se tomó la decisión de cambiar los valores nulos a 0 para que pueda ser utilizado dentro de nuestro análisis, sin eliminar todo el atributo.

Por otra parte, se revisaron los valores duplicados sin embargo, no hubo instancias ni id's en donde se identificara algún valor duplicado.

Para finalizar esta sección, se obtuvieron los rangos de los valores en donde se revisó el mínimo y máximo de cada columna numérica. Esto con propósito de analizar que los rangos dentro de la información sean razonables y también para verificar que no existiera ningún error de captura. Gracias a esto todos los valores

identificados dentro del dataframe eran consistentes con lo esperado, por ejemplo: age_years entre 0.2 y 16.0 años, weight_kg entre 2.0 y 70.6 kg, y las variables de comportamiento dentro de la escala de 0 a 10.

2.4 Transformación

Aunque se hicieron algunas transformaciones dentro del área de análisis y calidad de datos para mejorarlos. La única transformación estructural que se realizó dentro de este data frame fue la codificación de las variables categóricas. Como mencionado dentro de la sección 1.2 nuestro dataset consiste en 5 atributos categóricas las cuales son: Size, Breed_group, Intake_type, Medical_needs y Home_type. Lamentablemente ningún algoritmo de regresión lineal puede trabajar con datos tipo texto, por lo cual para poder realizar un análisis extenso incluyendo estas columnas se realizó una transformación para convertir la información categórica en varios atributos binarios independientes. No solo eso sino, que se ingresa un 1 si pertenecía al atributo y 0 en el caso de que no.

Gracias a esto nuestra transformación expandió nuestro dataset de 32 a 48 atributos, debido a que 5 de los atributos iniciales fueron reemplazados por 21 atributos binarios. La clasificación fue la siguiente: 4 atributos para size, 7 para Breed_group, 4 para Intake_type, 3 para medical_needs, y 3 para home_type. Sin embargo, las columnas adoption_id y return_reason no fueron modificadas debido a que no forman parte de las variables numéricas utilizadas en el modelado. El tratamiento de estas variables es descrito más adelante en la sección 5.1.

2.5 Carga

Para finalizar, se elaboró la última etapa del ETL, en donde se guardan todos los cambios realizados dentro del ETL. En otras palabras, se guardaron 42,000 instancias y 48 atributos. Las cuales 46 de ellas fueron

numéricas, listas para ser utilizadas en el modelo. Gracias al ETL se guardó en un nuevo archivo con el nombre de “dog_adoption_clean.csv” y va a ser la fuente de datos utilizados a partir de este punto, ya sea para el área de análisis o el modelaje.

2.6 Justificación

El ETL es un proceso sumamente importante en cualquier dataset, debido a que es una de las etapas que afecta la manera en la que se maneja la información de una manera más analítica. Sin un ETL las consecuencias se hubieran visto en etapas posteriores del proyecto ya sea en el EDA o PCA y afectaría en la manera en la que se ejecutarán.

Para empezar sin la modificación elaborada en la sección de transformación hubiera afectado en la manera en la que el modelo opera debido a que ningún tipo de modelo de regresión lineal ya sea elaborado sin frameworks o con puede operar sobre columnas de texto. Intentar entrenarlo sin esta transformación hubiera provocado errores de tipo antes de siquiera empezar el modelaje.

Para continuar, al dejar los valores nulos de `days_to_return` sin tratar hubiera propagado un NaN a cualquier cálculo posterior que incluyera este atributo. Por lo cual, se hubiera filtrado en etapas posteriores del modelo incluyendo PCA, en donde un solo valor nulo puede contagiar toda la matriz de covarianza haciéndola nula. Otra opción hubiera sido erradicar el 85% de las instancias por este atributo, sin embargo, la decisión de reemplazar el valor NaN con un 0 es mucho mejor debido a que nos ayuda a conservar nuestro dataset entero y de hecho es el valor correcto para ese atributo debido a que no hubo devolución para esos casos eso está demasiado implícito leyendo el dataframe.

Por otro lado, la revisión de los valores duplicados y los rangos numéricos fue importante debido a que aunque no hubo ningún cambio al data frame durante este

proceso generó confianza en los resultados obtenidos en las secciones posteriores, reflejando los patrones reales y no artefactos repetidos o incorrectos.

Para finalizar, un dataset de este tamaño es común incluso existen datasets mil veces más grandes en la vida real. Más grandes de lo que se está trabajando en este proyecto, por lo cual hacer las correcciones de manera manual tardaría bastante tiempo sin considerar los errores que alguien pudiera realizar, sin embargo, un proceso ETL programático y reproducible hace este proceso en segundos garantizando confiabilidad en los resultados.

3. Análisis Exploratorio de Datos (EDA)

3.1 ¿Qué es el EDA?

El análisis Exploratorio de datos también conocido como el EDA por sus siglas en inglés. Es un enfoque para analizar conjuntos de datos que consiste en resumir sus características principales, frecuentemente mediante métodos visuales antes de ser aplicados en cualquier modelo estadístico. El término fue definido por Jhon Tukey en 1977 en su libro, en donde describe al EDA como una forma de dejar que los datos hablen. Por otra parte, el EDA busca en describir patrones, detectar anomalías, definir suposiciones, y generar nuevas preguntas a partir de los datos obtenidos.[6]

En la práctica el EDA involucra examinar la distribución de cada variable, identificar los valores atípicos, los cuales podrían causar errores de captura o entender la composición de las variables categóricas, y medir la relación entre variables mediante correlaciones. De hecho durante esta sección se hablará de las distribuciones, los valores atípicos, variables categóricas y las correlaciones.

El EDA cumple con el propósito de ir más allá al describir los datos, ya que es el paso en donde informa decisiones posteriores del proyecto. La importancia del EDA es que nos ayuda a validar, descartar los datos y variables presentes dentro de nuestro proyecto, las cuales nos ayudan a construir nuestro modelo.

3.2 Distribuciones

El histograma de las variables numéricas principales se muestran en la figura 3.1 y 3.2. Los atributos que cabe destacar son la de Weight_in_kg, Age_years y Days_in_shelter, esto debido a que tienen distribuciones sesgadas hacia la derecha, en otras palabras se significa que la mayoría de los valores se concentran en la parte baja del rango. Esta distribución de las colas se puede interpretar como perros de mayor edad, mayor peso, o con estadías largas dentro del refugio. Por otra parte podemos observar como algunas de nuestras variables de comportamiento (Agresividad, Ansiedad, Energía, y Entrenamiento) muestran distribuciones un poco más uniformes. Sin embargo, cabe mencionar que aggression_score muestra el mismo sesgo hacia la derecha que los atributos anteriores refiriendo que hay bastantes perros que no exhiben aggression. Por otro lado, anxiety separation e incluso reactivity_to_dogs tienen una distribución más consistente a lo largo de su escala del 0 al 10. Previously_returned y neutered no muestran ninguna distribución debido a que son variables binarias, y la gran mayoría de los registros de previously_returned son 0 y 1 en neutered. A continuación se puede ver la figura 3.1.1 y 3.1.2 en donde se muestra la distribución de variables.

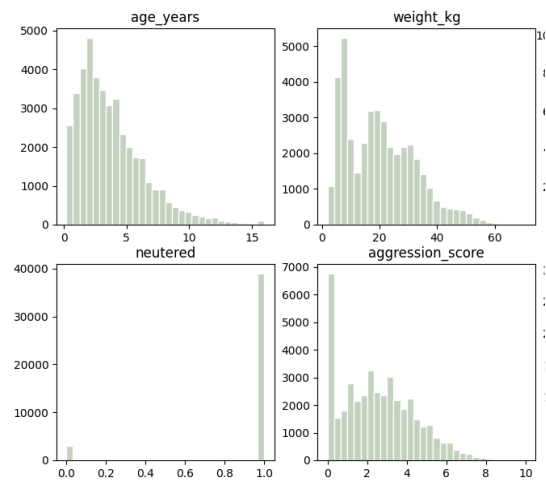


Figura 3.1.1- Distribución de las variables numéricas principales (Age_years, Weight_kg, returned, Agresion_score)

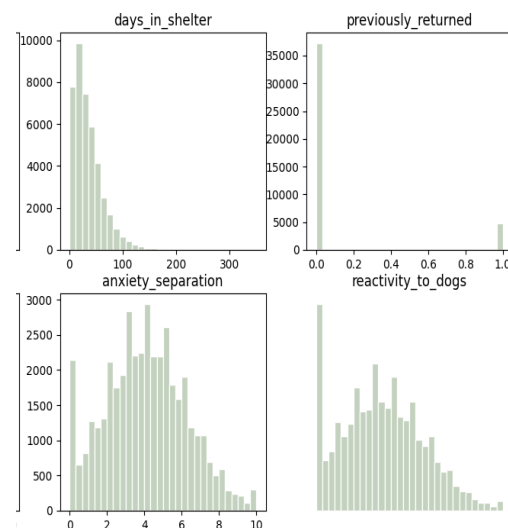


Figura 3.1.2 - Distribución de las variables numéricas principales (Days_in_shelter, Previously_returned, anxiety_separation, reactivity_to_dogs)

3.3 Valores atípicos

Para empezar, en esta sección se va a presentar los valores atípicos vistos en nuestro EDA, cabe mencionar que este tipo de valores son los datos que se alejan significativamente del resto de nuestro conjunto de datos. En este caso se planeó plantear Boxplots debido a que ayudan a visualizar estos datos de la manera más sencilla posible. Los atributos que se

graficaron fueron Age_years, Weight_kg y Days_in_shelter. Como se puede demostrar en la figura 3.3 muestra la confirmación de la presencia de los valores atípicos (outliers) en la parte superior de cada distribución. Por ejemplo, en Age_years podemos observar como el rango principal de edad está entre 2 a 4 sin embargo tenemos perros con edad hasta 16 años mostrando nuestros outliers, en el caso de Weight_kg tiene un rango hasta 70.6kg y un promedio de 20.27kg, y la Days_in_shelter tiene una estancia de hasta 350 días. Aunque sean outliers y se alejen de nuestro grupo de datos principal, se tomó la decisión de no eliminar ni recortar estos valores atípicos debido a que ninguno de estos valores representa un caso biológicamente implausible para un perro real. En el caso de remover estos datos o eliminarlos causaría un descarto de información real en lugar de eliminar el ruido.



Figura 3.2.1 - Valores atípicos Age_years

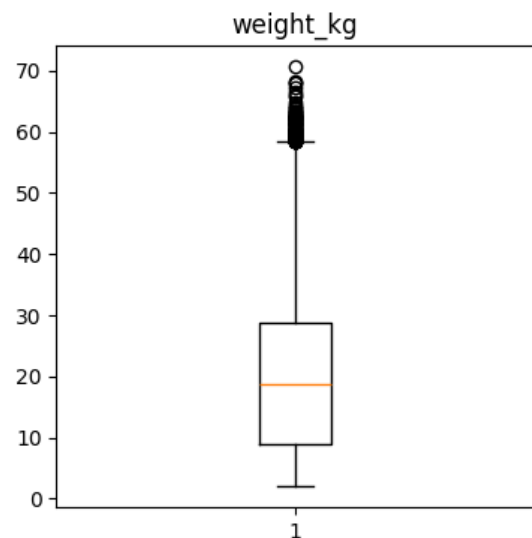


Figura 3.2.2 - Valores atípicos Weight_kg

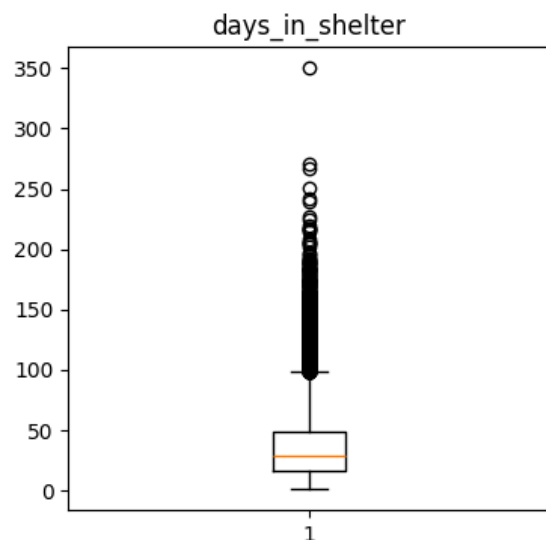


Figura 3.2.3 - Valores atípicos Days_in_shelter

3.4 Valores Categóricos

Como ya mencionamos en la sección 1.2 de este documento, nuestro dataset cuenta con variables categóricas: Size, Breed_group, Intake_type, Medical_needs y Home_type. En la figura 3.4 se muestra el desglose de conteo por categoría en donde muestra el balance de cada una de los atributos. Como podemos observar el atributo de Size está relativamente balanceado entre medium, small y large, por el contrario podemos observar como xlarge es claramente la minoría dentro de este atributo con alrededor de 3,400 instancias. Por otra

parte, en Breed_group podemos observar como está dominado por mixed con alrededor de 14,700 instancias, lo cual lo hace superar a los otros tipos de raza por casi el triple. Intake Type está dominado por stray, sin embargo, owner_surrender también representa una gran proporción de esta área. Por otro lado, en medical_needs podemos observar que está dominado por none con alrededor de 30,000 instancias superando las otras secciones por más de la mitad. Para finalizar, home_type está moderadamente inclinado hacia owned sobre apartamento o renta.

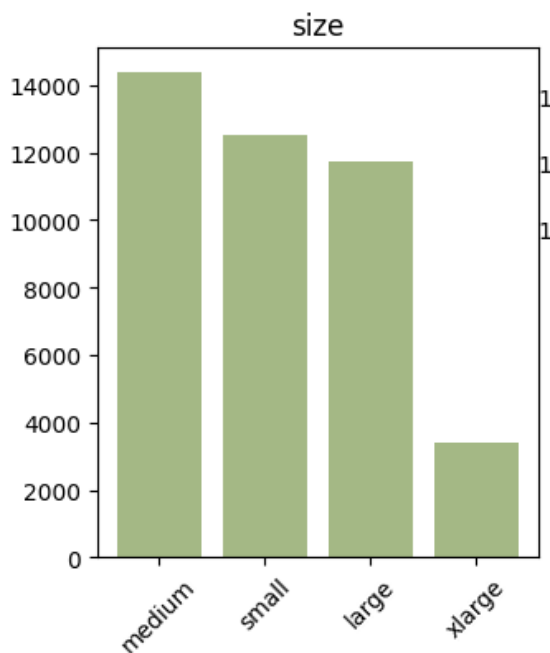


Figura 3.4.1 - Gráfica categórica (Size)

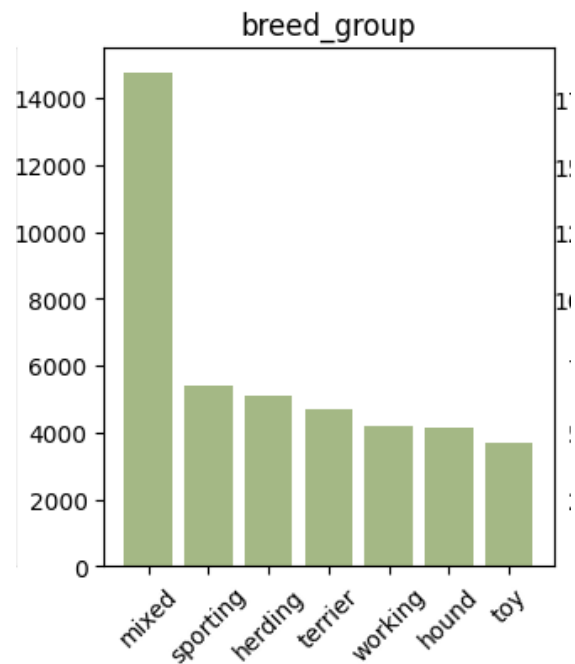


Figura 3.4.2 -Gráfica categórica (Breed_group)

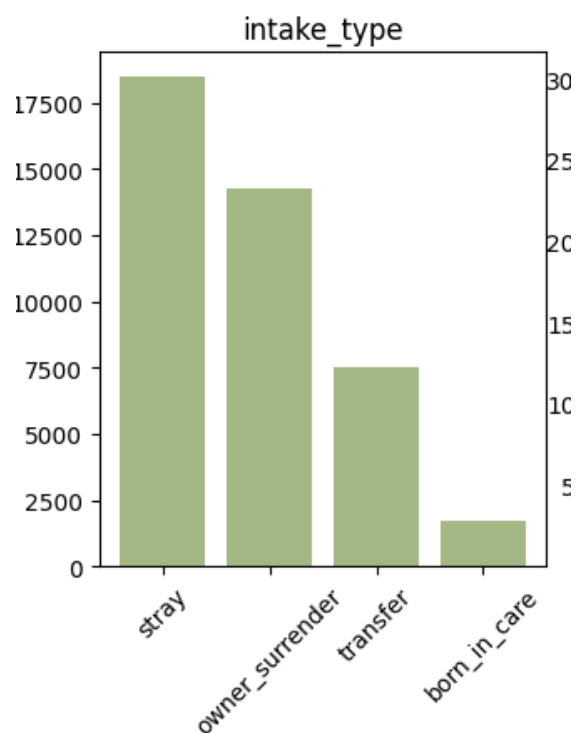


Figura 3.4.3 -Gráfica categórica (Intake_type)

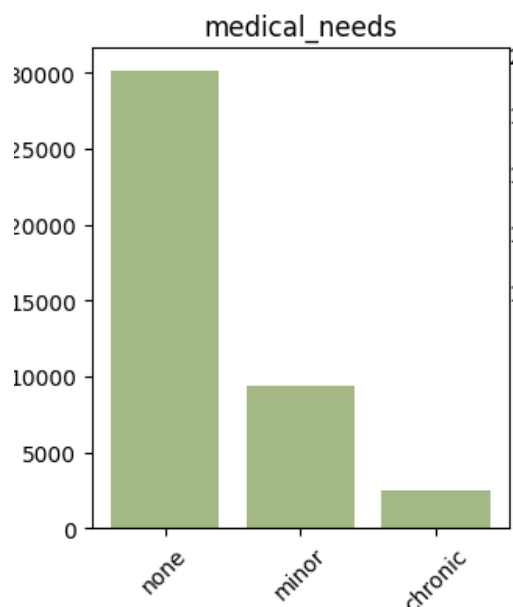


Figura 3.4.4 -Gráfica categórica (Medical_Needs)

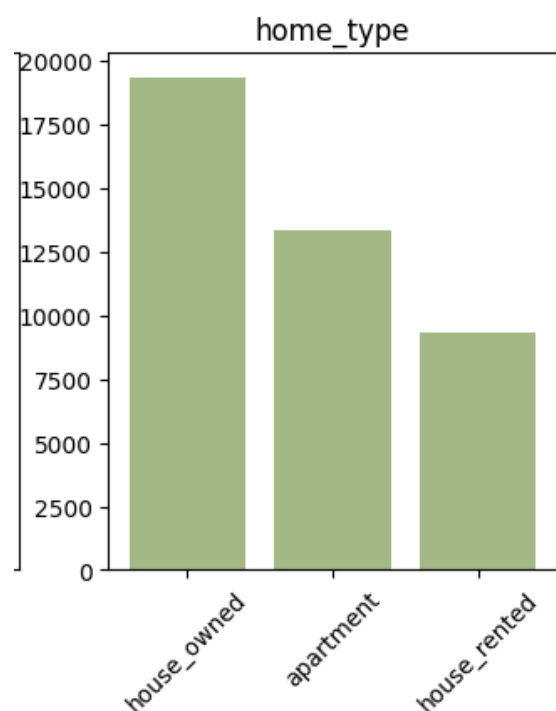


Figura 3.4.5 - Gráfica categórica (Home_type)

3.5 Correlaciones

En esta sección se elaboró una matriz de correlación entre variables, esto significa

que nos ayuda a visualizar si existe alguna correlación ya sea positiva o negativa entre variables. Esta matriz solo fue aplicada a las variables numéricas y durante esta sección fue en donde se verificó el hallazgo que motivó el cambio de la variable objetivo, el cual fue mencionado en la sección 1.3 Como se puede observar en la figura 3.5.1 podemos observar que ninguna variable de comportamiento tiene una correlación lineal relevante con days_in_shelter, la variable más alta es training_level con un valor de 0.23, el cual es una correlación débil que apenas explica ~5% de la varianza. Sin embargo, gracias a la matriz de correlación se pudo analizar otras correlaciones más notables como aggression_score (0.23), anxiety_separation (0.14), y reactivity_to_dogs (0.13), las cuales se asocian con la variable de returned. Es por eso, que estos atributos forman una relación más interesante que days_in_shelter, sin embargo está fuera del alcance de este proyecto de regresión sobre el peso. En la sección posterior se analiza la correlación entre size & weight kg. La cual es nuestra variable objetivo en nuestro proyecto.

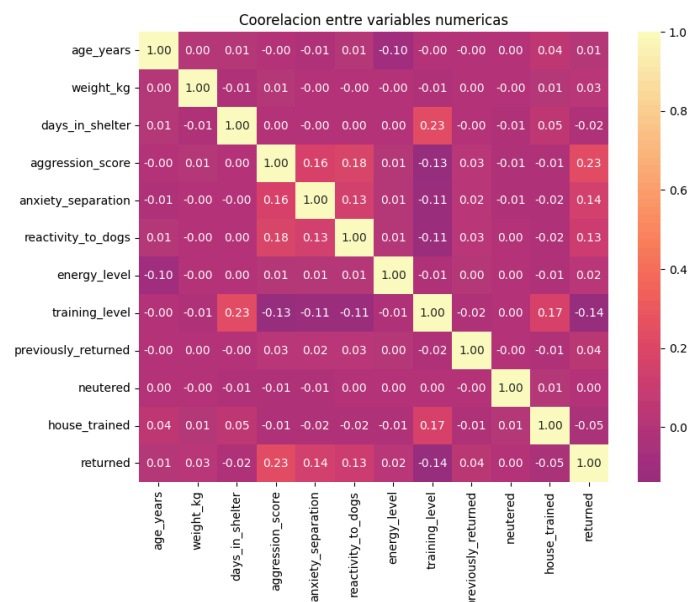


Figura 3.5 - Matriz de correlaciones entre variables numéricas.

3.6 Hallazgos clave y Justificación

Para clarificar la matriz de correlaciones de la sección 3.5 solo calcula las variables numéricas, por lo cual la variable de size no aparece en ella debido a que es una variable categórica. Sin embargo, durante la revisión de la sección 3.4 en donde hablamos de las variables categóricas, surgió una intuición a la hora de analizar los resultados obtenidos, esto se debe a que es sentido común que un perro que es clasificado como large o xlarge debería de pesar mucho más que un perro clasificado como small, esto se debe porque el tamaño y el peso están directamente conectados. Gracias a esto, al analizar que la mayoría de las correlaciones numéricas existentes tenían correlaciones débiles se decidió comparar weight_kg contra size. En este caso se realizó la comparación mediante un boxplot, esto se debe a que no se puede comparar una variable numérica con una categórica en la matriz de correlaciones. Por el contrario, un boxplot nos ayuda a comparar ambas variables sin importar su tipo.

En este caso el hallazgo más relevante fue la asociación que existe entre ellas demostrando que la hipótesis planteada era correcta de que a mayor tamaño corresponde un peso mayor y viceversa. En la figura 3.6.1 podemos observar las 4 categorías de tamaño muestran distribuciones de peso claramente separadas e incluso podemos ver una que otra que se traslapan entre sí. Como podemos observar small tienen alrededor de ~7kg por promedio, medium ~18kg, large~30kg, y xlarge con ~45kg.

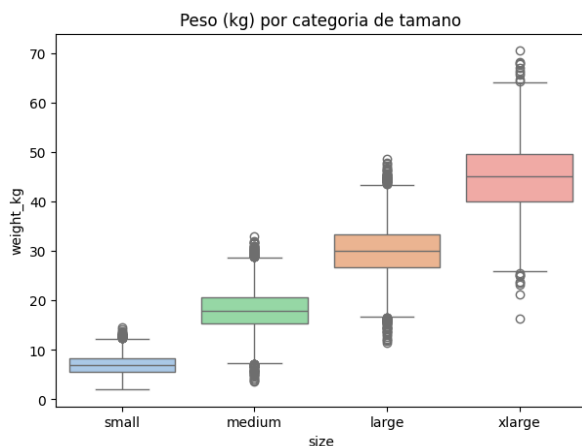


Figura 3.6.1 - Correlación entre tamaño y peso

4 Reducción de Dimensionalidad (PCA)

4.1 ¿Qué es el PCA?

El PCA o el análisis de componentes principales es una técnica que nos ayuda a la reducción de dimensionalidad que transforma un conjunto de variables numéricas, en un nuevo conjunto de variables no correlacionadas llamadas componentes principales. [7]

El objetivo principal del PCA no es solo eliminar la información, sino identificar las direcciones en las que los datos varían más. Todo esto con el propósito de representar un dataset con muchas columnas utilizando un número menor de componentes sin perder una proporción significativa de la información real.

El PCA se calcula a partir de la matriz de covarianza, mediante una descomposición en eigenvalores y eigenvectores. Los Eigenvectores representan la dirección de un componente principal, y sus eigenvalores representan la cantidad de varianza que ese componente explica. Al ordenar los eigenvalores de mayor a menor con sus eigenvectores correspondientes nos permite determinar cuántos componentes son necesarios para determinar un porcentaje de varianza total del dataset.

Durante la elaboración de este proyecto se aplicó el PCA sobre las variables numéricas del dataset, esto con el fin de explorar la posibilidad de reducción de la dimensionalidad, esto sin perder la información relevante para el modelado. Las secciones de 4.2 a 4.4 de este documento explican la metodología, los resultados, y la interpretación de este análisis.

4.2 Justificación

Gracias al área del ETL nuestro dataset cuenta con 46 columnas numéricas. Sin embargo, antes de pasar a la etapa del modelado se realizó un análisis de componentes principales (PCA) con el objetivo de entender cuánta de la dimensionalidad es realmente necesaria. En otras palabras, cuánta de esta información redundante existe dentro de nuestros 46 atributos. Cabe aclarar que, los resultados mostrados en la sección del PCA son solo con fines exploratorios, lo que se significa que el modelo 1.0 y modelo 2.0 no utilizan los componentes principales como variables de entrada.

4.3 Metodología

Siguiendo la estructura vista en clase y referenciada dentro del curso de PCA implementado con Numpy, el procedimiento fue el siguiente: para empezar se estandarizaron las 46 columnas numéricas a media 0 y desviación estándar 1. A continuación, se calculó la matriz de covarianza y al obtener los valores eigenvectores y eigenvalores, se ordenaron de mayor a menor y posteriormente se calculó el porcentaje de la varianza explicada por cada componente y se proyectaron los datos en un espacio de menor dimensión.

4.4 Resultados

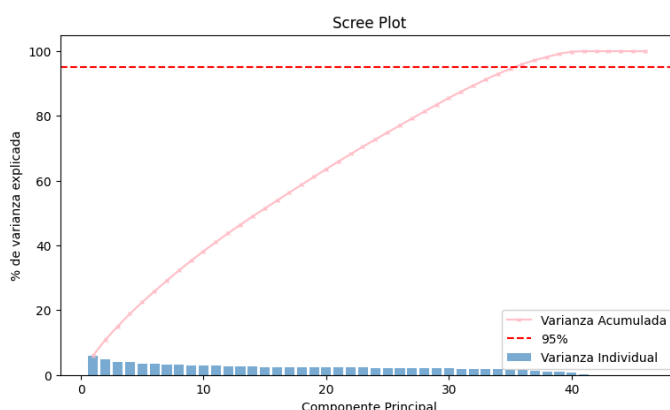
En nuestro dataset se puede mostrar que la varianza está distribuida de una manera uniforme entre los componentes. Gracias a la figura 4.4.1 podemos ver que nuestro primer componente explica apenas 6.04% de la varianza total y para poder alcanzar el umbral de 95% de la varianza explicada se necesitan 36 de los 46 componentes.

Figura 4.4.1 - Varianza explicada individual y acumulada.

Algo que cabe observar es que en los últimos componentes de PC42 a PC46 tenemos una varianza explicada de 0%, lo cual es un resultado esperado debido a que al conservar las 5 categorías categóricas (size, breed_group, intake_type, home_type y medical_needs) sin eliminar una categoría base, hace que estos cinco grupos de columnas sumen exactamente 1 en cada fila, por lo cual introducen una dependencia lineal exacta por grupo. Gracias a PCA podemos identificar como ciertos componentes como los descritos hace rato presentan una varianza prácticamente nula.

4.5 Interpretación

Aunque en la sección anterior pudimos ver como PC1 y PC2 explican poca varianza, se elaboró una gráfica (Figura 4.5.) para observar visualmente las agrupaciones por tamaño, las cuales están organizadas en un patrón de franjas. Los patrones de franjas son una firma característica en cuanto a las variables categóricas codificadas como binarias, en contraste si se estuviera graficando las variables numéricas se mostrará en forma de nube de puntos sin bordes definidos. Gracias a esto, podemos confirmar que size es la fuente de variación más estructurada del dataset, lo cual consiste con el hallazgo de la sección 3.6, obtenida de forma completamente independiente durante el EDA.



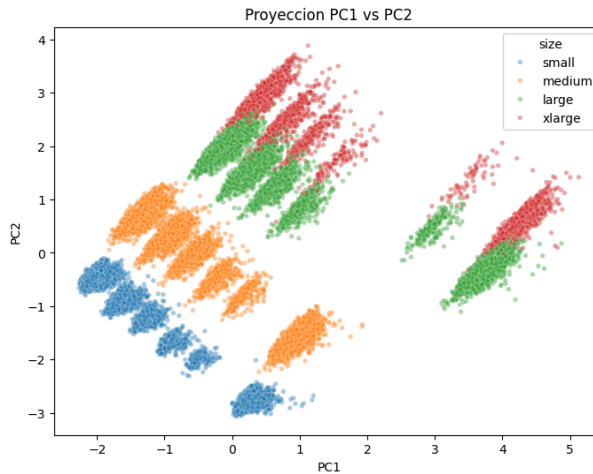


Figura 4.5.1 - Proyección de datos sobre los dos primeros componentes principales.

5 Modelo 1.0: Regresión lineal sin framework

5.1 Preparación de datos

Para la preparación de datos se limitaron algunos features, como por ejemplo cualquier variable relacionada con el proceso de adopción o con eventos posteriores, debido a que si estos se habrían introducido fuga de información, debido a que no estarían disponibles al momento de hacer la predicción. Gracias a esto se limitó a 27 features, con 6 de ellas siendo numéricas continuas (edad, agresividad, ansiedad por separación, reactividad, energía, y entrenamiento) 3 binarias (previamente devuelto, esterilizado, entrenado en casa) y 18 columnas one hot de las categorías de (size, breed group, intake type, y medical needs). Por otra parte, decidimos dividir nuestro modelo en 60% entrenamiento lo que significa 25,200 filas, el 20% en pruebas que son 8,400 filas y el 20% de validación con 8,400. En el caso de las 6 numéricas continuas se normalizaron utilizando únicamente la media y la desviación estándar del conjunto de entrenamiento, todo esto con el fin de evitar fuga de información del conjunto de pruebas hacia el proceso escalado.

5.2 Descripción del modelo

En este caso el modelo implementado corresponde a una regresión lineal múltiple, esto significa que es una técnica de estadística para predecir el valor de una variable dependiente utilizando dos o más variables independientes. [8]. Una regresión lineal múltiple se expresa como: $\hat{y} = Xw + b$, en donde cada predicción resulta en una combinación lineal ponderada por las 27 features utilizadas más el bias o término independiente. Tanto la función de hipótesis como la función de costo y el algoritmo de optimización se implementaron desde cero, sin recurrir a ninguna librería de aprendizaje automático. Sin embargo, el uso de Numpy y Pandas si fue utilizado para la implementación únicamente como herramientas de álgebra lineal y manejo de datos, no para resolver el algoritmo de regresión en sí.

5.3 Función de hipótesis

La función de hipótesis se define como $\hat{y} = h(x) = Xw + b$, en donde X representa la matriz de los features de un conjunto de perros y el w es el vector de pesos (weight) aprendidos para cada una de las 27 variables. En Matemáticas y en el curso de Machine Learning representado por Andrew Ng menciona como la función de hipótesis simplemente calcula una combinación lineal [9]. En otras palabras esto significa, que cada feature es multiplicado por su peso correspondiente, se suman los resultados y se añade el bias (término independiente), así produciendo una predicción numérica del peso del perro. Gracias a esta función nos ayuda a traducir los datos de entrada en una predicción real, sin ella no habría ninguna forma de convertir las características del perro en un número interpretable.

5.4 Función de costo (MSE)

La función de costo utilizada es el error cuadrático medio (MSE), la cual está definida como $J(w) = (1/n)\sum(\hat{y}_i - y_i)^2$, en

donde n es el número de observaciones en el conjunto de entrenamiento. El propósito de esta función es calcular qué tan alejadas están las predicciones del modelo ($\hat{y}$) de los valores reales (y), elevando al cuadrado para penalizar los errores grandes y mantener la función matemáticamente conveniente. Gracias a esta función nos ayuda a que nuestro modelo tenga forma de saber si está prediciendo bien o mal, es por eso que esta métrica la que el algoritmo de optimización busca minimizar en cada iteración.

5.5 Algoritmo de Optimización (Descenso de Gradiente)

El algoritmo de optimización utilizado en este caso es el descenso de gradiente por lotes, cuya regla de actualización es de $w = w - \alpha \nabla J(w)$, en donde el gradiente se calcula como $\nabla J(w) = (\frac{2}{n}) X^T (\hat{y} - y)$.

α es la tasa de aprendizaje o learning rate. En cada iteración (EPOCH), este algoritmo calcula la dirección en la que el error crece más rápido con respecto a cada peso y mueve los pesos en la dirección contraria así reduciendo de manera gradual el valor de la función de costo. Gracias a este algoritmo es el que ayuda a entrenar al modelo, debido a que es un mecanismo iterativo que parte los pesos desde 0, y los va ajustando poco a poco hasta llegar a los valores en donde minimiza el error del modelo.

5.6 Entrenamiento

En este caso nuestro modelo se entrenó con una tasa de aprendizaje de 0.15 con 400 épocas (EPOCHS). Cabe mencionar, que los hiperparametros se eligieron de forma experimental, esto significa que al utilizar una tasa de aprendizaje alta generaba colisiones en el MSE de entrenamiento, por el contrario una tasa menor requiere mucho más épocas para poder converger. Al analizar y correr el modelo varias veces se decidió utilizar 0.15 debido a que resultó ser un valor que tenía una

convergencia rápida y estable. En este caso como mencionado en la sección 5.1 los datos se dividieron en 60% para entrenamiento (25,200) filas, 20% para validación (8,400) y 20% para las pruebas (8,400). Cabe mencionar que el conjunto de validación como el de prueba se mantuvieron completamente fuera del ajuste de parámetros con el propósito de garantizar una evaluación imparcial del rendimiento del modelo ante datos totalmente nuevos y poder evitar el sobreajuste (overfitting). Al correr nuestro modelo el MSE de entrenamiento bajó de 144.88 en la primera época a 17.74 en la época 400, lo que significa que se estabilizó alrededor de la época 150 a la 200 y tuvo una reducción aproximadamente del 87.7%. En la figura 5.6.1 podemos observar como el modelo efectivamente convergió.

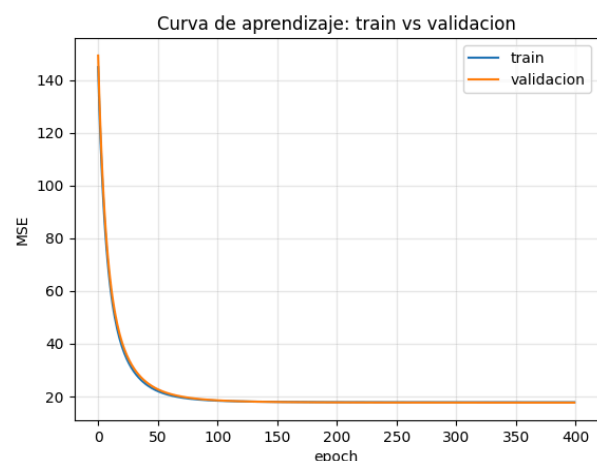


Figura 5.6.1 - Curva de aprendizaje de train contra validación.

5.7 Resultados

Debido a que hubo tres conjuntos de datos, los resultados obtenidos se pueden resumir en la siguiente tabla.

Métrica	Entrenamiento	Validación	Prueba
MSE	17.744	17.604	17.481

RMSE (kg)	4.212	4.196	4.181
MAE (kg)	3.149	3.108	3.124
R ²	0.8796	0.8842	0.8794

Gracias a esta tabla podemos explicar los resultados obtenidos de una manera más consistente. Para empezar tenemos un RMSE de aproximadamente 4.2 kg, lo cual se significa que en promedio la predicción se desvía del peso real en poco más de 4 kg, esto debido a que el peso promedio de los perros dentro de la base de datos es de alrededor de 20.27kg (Sección 3.3), este error representa cerca de un 20% del peso promedio. Por poco más de 3 kg por predicción. Por otro lado tenemos un R² de 0.88, lo cual se significa que el modelo logra explicar el 88% de la variabilidad del peso de los perros a partir de las 27 features utilizadas.

Un aspecto que cabe mencionar es como las tres métricas de error y el R² son muy parecidas entre entrenamiento, validación y pruebas. En sí la mayor diferencia entre los tres conjuntos es de alrededor de 0.140 en MSE. Gracias a esto podemos concluir que el modelo no sufre de varianza alta (Overfitting). Overfitting es cuando el modelo memoriza el conjunto de entrenamiento en lugar de aprender un patrón generalizable, lo cual resulta en un error notablemente menor en entrenamiento que en validación y prueba, algo que es visto que no ocurre en el modelo. Este hallazgo junto con el análisis de sesgo se retoman con mayor detalle en la sección 5.9.

5.8 Interpretaciones

Al observar los coeficientes con mayor magnitud del modelo se confirma que las categorías de tamaño dominan la predicción `size_xlarge` (+23.01), `size_small` (-14.82) y `size_large` (+8.13) tienen un impacto considerablemente mayor que

cualquier otro feature, seguidas de lejos por `medical_needs` e `intake_type`. Gracias a esto podemos verificar la consistencia que existe con lo observado desde el EDA y PCA, lo cual responde a la hipótesis de que el tamaño del perro es por mucho el valor más determinante de su peso.

En la figura 5.8.1 se muestra el patrón observado de peso real contra el peso predicho, en donde las predicciones se agrupan en bandas horizontales relativamente planas en lugar de distribuirse de manera suave a lo largo de la diagonal. Esto se debe a que cada perro pertenece exactamente a una categoría de tamaño y esa categoría por sí sola ya explica la mayor parte del peso esperado, por el contrario el resto de los features (edad, agresividad, nivel de energía, etc) solo desplazan la predicción ligeramente dentro de esa banda. En otras palabras, esto significa que el modelo predice con bastante precisión a qué rango de peso pertenece el perro, pero no puede afinar tan bien la posición exacta dentro de ese rango solo utilizando las variables de comportamiento.

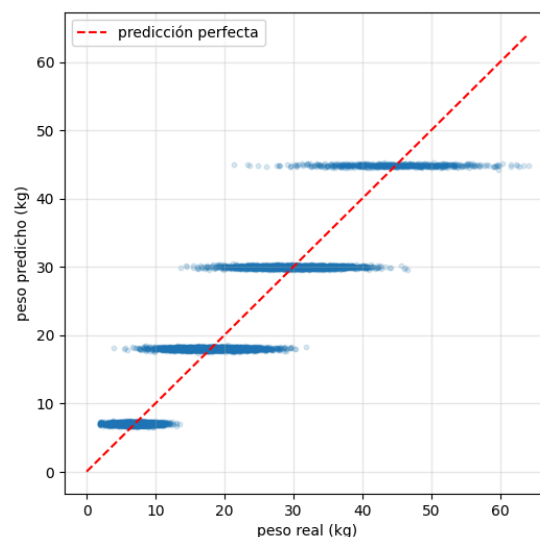


Figura 5.8.1 - Gráfica de predicción

Por otra parte, el sesgo del modelo se comparó su desempeño contra un modelo de referencia que simplemente predice el

promedio del peso para todos los perros, esto sin utilizar ningún feature. El modelo de referencia obtuvo un MSE de 151.999 en el conjunto de validación, por otra parte nuestro modelo alcanzó un MSE de apenas 17.604 en el mismo conjunto, lo cual representa una reducción de más del 88% en el error. Gracias a esto podemos concluir que es una diferencia tan amplia respecto a un modelo que no aprende nada, lo cual indica que el sesgo de nuestro modelo es bajo, en otras palabras significa que el modelo si logra capturar una relación real entre los features disponibles y el peso del perro.

Por otra parte en el área de el R^2 tiene una estabilización en el punto 0.88 y esto hace que no se acerque más a 1. Sin embargo, vale la pena mencionar que la R^2 no es evidencia de que nuestro modelo contenga underfitting. En este caso el underfitting implicaría un modelo más complejo utilizando las mismas features, podría lograr un ajuste sustancialmente mejor. Para verificar esta interpretación, se calculó la varianza del peso real dentro de cada categoría de tamaño, sin considerar cualquier otro feature. Small obtuvo una varianza de 3.97, medium de 15.91, large de 25.13 y xlarge de 49.74. Al promediar estas varianzas ponderado por el número de perros por cada categoría, obtenemos una varianza residual esperada de 17.622, lo cual es prácticamente idéntica al MSE real del modelo en el conjunto de validación (17.604). Esto confirma que el ~12% de varianza no explicada corresponde a variación natural del peso dentro de cada categoría de tamaño y no a información recuperable mediante features adicionales o un modelo más complejo.

5.9 Diagnóstico final

Gracias a nuestro modelo combinando los resultados de las secciones anteriores, el modelo de regresión lineal 1.0 presenta un sesgo bajo, debido a que el R^2 de entrenamiento (0.8796) está por encima del que obtendría un modelo trivial que

simplemente prediga el peso promedio para todos los perros (baseline), cuyo MSE de validación es de 151.999 frente a los 17.604 del modelo entrenado, lo cual menciona que hubo una reducción del error superior al 88% . Esto indica que el modelo si está capturando la relación entre las variables y el peso, y no se está quedando corto por falta de capacidad.

Por otra parte, podemos mencionar que la varianza también es baja debido a que el desempeño del modelo es prácticamente idéntico entre los tres conjuntos. Esto se puede demostrar en como la R^2 de entrenamiento fue de 0.8796, 0.8842 en validación y 0.8794 en pruebas, también se puede observar cómo se obtuvo un MSE de 17.744, 17.604 y 17.481 respectivamente. Gracias a estos resultados nos ayuda a definir una conclusión en donde el modelo generaliza bien a los datos que no vio durante el entrenamiento, esto significa que el modelo no memorizo el conjunto de entrenamiento, debido a que si lo hubiera hecho el error dentro de validación y en pruebas sería notablemente mayor que en el de entrenamiento.

Gracias a que tenemos un sesgo bajo y una varianza baja podemos llegar a la conclusión de que el modelo se encuentra en un punto de buen ajuste también conocido como fit. Tampoco muestra señales de overfitting o underfitting, obviamente no hay overfitting porque el modelo explica una proporción alta y estable de la varianza del peso (~88%) en los tres conjuntos, y como se mostró en la sección 5.8 el ~12% del error restante coincide con la variación natural de peso que existe dentro de cada categoría de tamaño, en otras palabras es decir que el ruido irreductible es del propio fenómeno y no información que le modelo está dejando de aprovechar. Por otra parte, no hay overfitting debido a que no muestra una brecha clara entre el error de entrenamiento y validación, sin embargo en nuestro modelo la brecha es mínima de 17.744 frente a 17.604.

Por esta razón, la búsqueda de regularización confirmó que ningún valor de λ mayor a cero mejora el desempeño de validación, en otras palabras el mejor resultado se obtiene precisamente con $\lambda = 0$, lo cual consiste con nuestro diagnóstico de un varianza baja, debido a que la regularización existe para corregir el sobreajuste y al no haber algún sobreajuste que corregir, penalizar los pesos del modelo solo le resta la capacidad de ajuste sin ningún beneficio en generalización.

6 Modelo 2.0: Regresión lineal con framework

6.1 Contexto

Una vez validado el funcionamiento del modelo 1.0 implementado desde 0 mediante descenso de gradiente manual, se desarrolló el segundo modelo utilizando la librería scikit-learn, uno de los frameworks de aprendizaje automático más utilizados en Python para problemas de regresión y clasificación [3]. El propósito de este segundo modelo es validar de manera independiente los resultados obtenidos manualmente, comparando ambas implementaciones sobre exactamente el mismo conjunto de datos, la misma participación en entrenamiento, validación y pruebas, y las mismas variables predictoras. Por otro lado, también era para aplicar de manera formal una técnica de regularización, en este caso se utilizó Ridge (Regularización L2), evaluando si esta permite mejorar el desempeño del modelo en el conjunto de validación.

A diferencia del modelo 1.0, en donde el término de sesgo debía incorporarse manualmente como una columna adicional de unos dentro de la matriz de variables, en scikit-learn calcula este término de manera automática al ajustar el modelo, gracias a eso no fue necesario replicar este paso. Otro ejemplo es como el modelo 1.0 tenía la actualización de los parámetros de forma iterativa a lo largo de 400 épocas mediante descenso de gradiente, el algoritmo de regresión lineal de scikit-learn resuelve el problema de manera directa, mediante una solución analítica de mínimo de cuadrados, sin requerir un proceso iterativo.

Haber mantenido las condiciones idénticas de experimentos permite atribuir cualquier diferencia observada entre ambos modelos exclusivamente al método de entrenamiento y no a variaciones externas de los datos utilizados. Esta comparación controlada es la que nos permite más adelante en la sección de este artículo confirmar la validez del modelo 1.0 y evaluar de forma rigurosa el efecto real de la regularización sobre el desempeño del modelo.

6.2 Entrenamiento

El entrenamiento del modelo 2.0 se realizó utilizando la clase de `LinearRegression` de scikit-learn, ajustada sobre el mismo tipo de conjuntos de entrenamiento definido previamente (25,200 observaciones corresponde al 60% del dataset), con las variables numéricas estandarizadas mediante media y desviación estándar calculadas exclusivamente a partir de dicho conjunto de entrenamiento y posteriormente aplicadas sin modificación a los conjuntos de validación y pruebas.

A diferencia de una implementación por descenso de gradiente, scikit-learn resuelve el problema de regresión lineal mediante una solución analítica de mínimos cuadrados. En otras palabras, esto significa que en lugar de ir ajustando los parámetros de manera iterativa a lo largo de múltiples épocas, este se calcula de manera directa los coeficientes que minimizan el error cuadrático medio sobre el conjunto de entrenamiento en un solo paso. Gracias a esto elimina la necesidad de definir los hiperparámetros como es la aprendizaje o el número de épocas de entrenamiento, no solo esto sino que también garantiza que la solución obtenida sea la más óptima para el conjunto de datos y para las variables utilizadas, esto sin depender de la convergencia de un proceso iterativo.

En este caso el modelo 2.0 se entrenó sobre el conjunto de entrenamiento reservando el conjunto de validación para evaluar su capacidad de generalización y el conjunto de prueba para la evaluación final. Este modelo sigue el mismo esquema de participación que el modelo 1.0 el cual es de 60/20/20.

6.3 Resultados

Tras el entrenamiento del modelo 2.0 a continuación se puede observar los tres conjuntos de datos obtenidos.

Métrica	Entrenamiento	Validación	Prueba
MSE	17.744	17.600	17.478
RMSE (kg)	N/A	N/A	4.181
MAE (kg)	N/A	N/A	3.124
R ²	0.8796	0.8842	0.8794

Gracias a estos resultados podemos observar que el modelo explica aproximadamente el 88% de la varianza del peso de los perros en los tres conjuntos, esto se debe a que tiene un R² entre 0.8794 y 0.8842, con un error promedio de 3.124kg sobre el conjunto de prueba, en otras palabras, en promedio las predicciones se desvían un poco más de 3 kg del peso real.

La figura 6.2.1 muestra la relación entre el peso y el peso predicho sobre el conjunto de prueba. Al igual que el modelo 1.0 se puede observar como las predicciones se agrupan en bandas horizontales concentradas alrededor de valores específicos. Esto es a través de distribuirse de forma continua a lo largo de la línea de predicción perfecta. Lo que podemos observar de este patrón es que refleja la fuerte influencia que ejercen las variables de tamaño sobre las predicciones, perros pertenecientes a la misma categoría de tamaño reciben predicciones muy similares entre sí, independientemente de las otras características individuales, lo cual hace que se concentren de manera visual las predicciones en bandas asociadas a cada categoría.

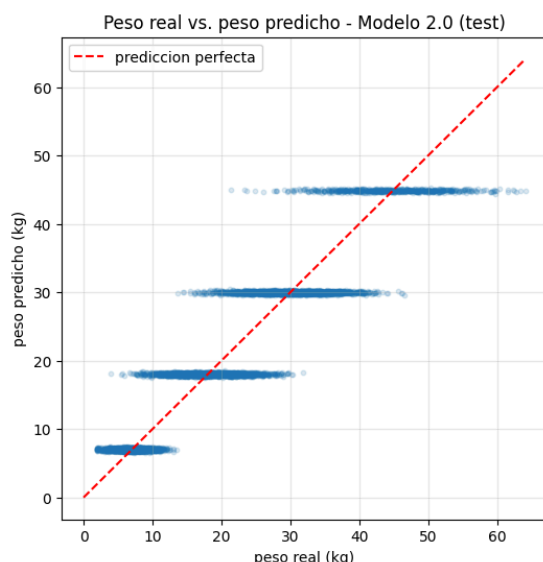


Figura 6.2.1 Predicción de modelo 2.0

6.4 Regularización

En este caso se aplicó una técnica de regularización sobre el modelo 2.0 utilizando Ridge (Regularización L2), la cual está disponible dentro de la librería de scikit-learn. Esta técnica agrega una penalización proporcional al tamaño de los coeficientes del modelo dentro de la función que se minimiza durante el entrenamiento, controlado por un hiper parámetro llamado alpha. De hecho, entre más grande el valor del alpha sea, más se penaliza los coeficientes grandes así empujándolos hacia valores más pequeños con el fin de reducir la sensibilidad del modelo a los datos de entrenamiento y así mismo mejorando su capacidad de generalización.

Para poder determinar si la técnica aporta alguna mejora se entrenó un modelo Ridge distinto para cada uno de los valores de alpha. Así evaluando en cada caso el MSE únicamente con el conjunto de validación. Los resultados de esto se puede mostrar en la siguiente tabla:

Alpha	MSE de validación
0.1	17.600
1.0	17.602
5.0	17.608
10.0	17.616
25.0	17.644
50.0	17.699
100.0	17.837
250.0	18.453
500.0	19.970

Como podemos observar en la tabla el error de validación es mínimo comparado con el valor más pequeño de alpha y aumenta de manera progresiva conforme se va incrementando la fuerza de regularización, hasta alcanzar el 19.970 con un alpha de 500,

un valor notablemente peor que el obtenido sin regularizar (17.600). Si observamos en la figura 6.4.1 podemos ver la curva del error de validación la cual es prácticamente plana para valores bajos de alpha y va creciendo de forma un poco más pronunciada a partir del alpha 100.

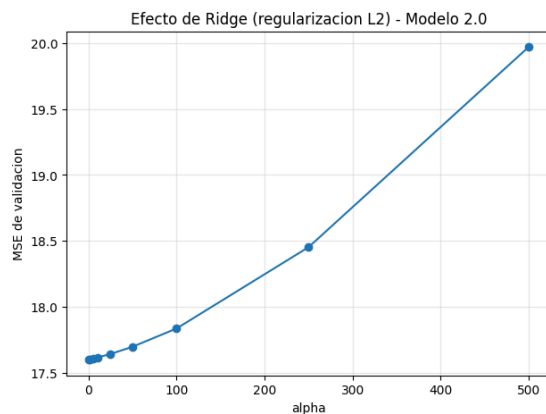


Figura 6.4.1 Efecto Ridge

Cabe mencionar que el mejor resultado corresponde a alpha de 0.1 el cual básicamente es el modelo sin ninguna forma de regularizar. Al evaluar el modelo sobre el conjunto de pruebas se obtuvo un MSE de 17.478, RMSE de 4.181, MAE de 3.124 y R^2 de 0.8794, los cuales son valores idénticos a los del modelo 1.0 el cual no tiene regularización. Esos datos se pueden encontrar en la sección de resultados.

Gracias a este resultado podemos verificar que la regularización no aporta una mejora al desempeño del modelo. Esto era de esperarse en cierta manera debido a que Ridge se utiliza para problemas de alta varianza, así penalizando a los coeficientes que se han vuelto excesivamente grandes por ajustarse demasiado a las particularidades del conjunto de entrenamiento. Como fue visto en el modelo 1.0 y en este el modelo no presenta varianza alta, lo cual hace que no exista un sobreajuste que la regularización pueda corregir, lo unico que hace es que le reste capacidad de ajuste al modelo lo cual explica el deterioro progresivo del desempeño conforme alpha aumenta.

6.4.2 Regularización bajo condiciones de sobreajuste inducido

Con el fin de tener un mejor entendimiento de la regularización se creó un escenario en donde existirá el sobreajuste dentro del modelo. En este caso se generaron las variables polinomiales de 3r grado a partir de las 6 variables numéricas originales (incluyendo sus términos al cuadrado, al cubo y sus interacciones), gracias a esto el total de variables pasa de 27 a 104. En este caso el modelo solo se entrenó utilizando 300 observaciones del conjunto de entrenamiento en lugar de utilizar los 25,000 disponibles, lo que genera una relación desfavorable entre la cantidad de parámetros a ajustar y cantidad de datos disponibles.

Bajo estas condiciones el modelo mostró un claro sobreajuste debido a que en los resultados se observó un MSE de entrenamiento de 12.670 ($R^2 = 0.9179$) frente a un MSE de validación de 30.712 ($R^2 = 0.7979$), esto con una brecha de 18.043, lo cual lo hace sustancialmente mayor a la brecha original del modelo 2.0 en donde era de -0.143. Gracias a esto nos da información de que el modelo memorizó particularidades del conjunto de entrenamiento en vez de aprender.

Gracias a estos resultados se abrió la oportunidad de aplicar la regularización a este escenario, así evaluando los mismos valores de alpha utilizados anteriormente. En este caso gracias al cambio de complejidad dentro del modelo abrió nuevas posibilidades de resultados que no se vieron en el modelo original. En este caso el menor MSE de validación fue de 30.348 con un alpha de 1, lo cual estuvo debajo del valor de 30.652 con alpha de 0.1, lo cual significa que el modelo si llego a mejorar con la regularización en este caso. Otro aspecto visto en estos datos es que el modelo otra vez empieza a decaer a partir de un alpha de 1, lo que significa que llegó a su punto óptimo. En la siguiente tabla podemos observar el alpha con su MSE de validación.

Alpha	MSE de validación
0.1	30.652
1.0	30.348

5.0	32.154
10.0	37.218
25.0	53.594
50.0	73.498
100.0	95.506
250.0	119.924
500.0	132.773

Una vez finalizado este proceso se realizó otro proceso de regularización utilizando Ridge pero en este caso se quería observar si había otro componente de alpha que se acercara más que el de 0.1. Para que esto fuera posible en este caso en vez de inicializar alpha con valores altos, se prefiere realizar con valores más cercanos a donde se encontró el valor más pequeño de MSE. Como podemos observar en la siguiente tabla aunque tenemos un valor de alpha de 0.5 sigue siendo mayor que el alpha de 1. Lo que significa que aunque hagamos un acercamiento a este alpha, sigue siendo el mejor valor de MSE.

Alpha	MSE de validación
0.1	30.652
0.5	30.466
1.0	30.348
2.0	30.415
3.0	30.792
5.0	32.154
70.0	34.004
10.0	37.218

Por otra parte, en la figura 6.4.1.1 muestra la relación entre MSE de entrenamiento con el de validación. En este caso podemos observar como cuando el MSE de entrenamiento aumenta de forma continua a medida que el alpha va creciendo, el MSE de validación desciende ligeramente y

posteriormente aumenta, formando una curva, lo cual es el punto óptimo de regularización.

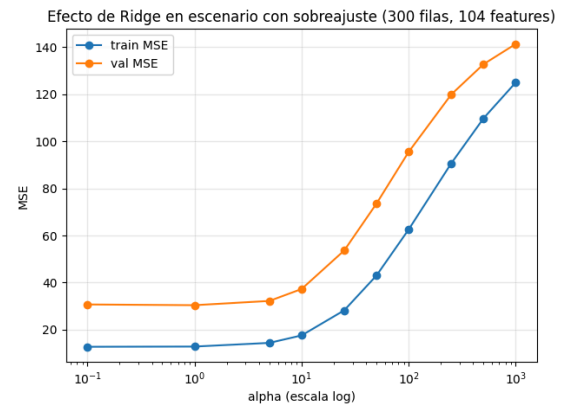


Figura 6.4.1.1 - Relación entre MSE de entrenamiento y validación.

Gracias a este experimento mejora y enriquece el análisis de esta sección debido a que confirma que la regularización L2 si mejora el desempeño del modelo, sin embargo, únicamente cuando existe una varianza alta (sobreajuste) que se quiera corregir.

6.5 Diagnóstico final

Gracias a los resultados obtenidos, podemos concluir que el modelo 2.0 presenta un sesgo bajo debido a que su R^2 de entrenamiento tiene 0.8796, el cual está por encima de un modelo que simplemente prediga el peso promedio para todos los perros (baseline), en el cual el MSE de validación es de 151.999 frente a los 17.600 del modelo entrenado, eso básicamente es una reducción del error superior al 88%. Lo que significa que el modelo logra capturarla relación real entre las variables disponibles y el peso de los perros sin quedarse corto por falta de capacidad de ajuste.

Por otro lado la varianza también resulta baja, debido a que el desempeño del modelo es prácticamente idéntico en los tres conjuntos de R^2 . No solo eso sino, que también tiene una diferencia entre el error de validación y el entrenamiento de -0.143. Esto significa que el modelo generaliza bien los datos que no vio durante su entrenamiento, sin señales de haberse ajustado en exceso a las particularidades del conjunto de entrenamiento.

Con un sesgo bajo y varianza baja el modelo 2.0 se encuentra en un buen punto de ajuste (fit), sin evidencia de overfitting o underfitting. Tras la extensión de complejidad en el modelo 2.0 se tuvo la posibilidad de aplicar Ridge con distintos valores de alpha para así poder tener un mejor desempeño con el valor más bajo, al aplicar el experimento hablado en la sección 6.4.2 se demuestra como la regularización llega a ser efectiva solo si el modelo cuenta con una varianza alta, de lo contrario no existe el margen de mejora sino al contrario. Gracias a esto podemos concluir que tenemos varianza baja debido a que la regularización L2 existe para corregir el sobreajuste, como fue mencionado en la sección anterior.

7 Comparación entre modelos

Una vez al haber evaluado ambos modelos de manera independiente, podemos analizarlos y hacer una comparación de manera directa. Gracias a que ambos modelos fueron entrenados con el mismo dataset, la misma partición en entrenamiento, validación y prueba (60/20/20) y las mismas variables predictoras. Podemos observar la siguiente tabla la cual resume las métricas obtenidas por cada uno.

Métrica	Modelo 1.0	Modelo 2.0	Modelo 2.0 con regularización
Train MSE	17.744	17.744	12.761
Train R2	0.8796	0.8796	0.9173
Validación MSE	17.604	17.600	30.348
Validación R2	0.8842	0.8842	0.8003
Prueba MSE	17.481	17.478	29.232
Prueba RMSE	4.181	4.181	5.407
Prueba	3.124	3.124	4.087

MAE			
Prueba R2	0.8794	0.8794	0.7983
Mejor hyperparámetro de regularización	0	0.1	1

Como fue observado en la tabla ambos modelos son prácticamente idénticos, lo único que los diferencia son el orden de milésimas en el error de validación y prueba. Obviamente este es un resultado esperado debido a que ambos modelos resuelven el mismo problema de optimización sobre los mismos datos, en realidad, la única diferencia real entre ellos es el método utilizado para llegar a la solución. Por ejemplo, el modelo 1.0 se ajusta de manera iterativa mediante descenso de gradiente a lo largo de 400 épocas, por el contrario el modelo 2.0 resuelve el problema de manera directa mediante una solución analítica de mínimos de cuadrados.

Por otra parte, el diagnóstico de sesgo y de varianza también fue consistente entre ambos modelos debido a que ambos presentan un sesgo bajo, esto con respecto al modelo de referencia trivial (MSE de validación de 151.999 frente a ~17.6 en ambos modelos) y varianza baja, evidenciada por la similitud entre sus métricas de entrenamiento, validación y pruebas. En ambos casos, esto los sitúa en un punto de buen ajuste en donde no encuentran evidencia de underfitting o overfitting.

Para finalizar, la diferencia más grande entre ambos modelos es dentro del análisis de regularización. En su configuración original de ambos modelos ninguno de ellos logró mejorar el desempeño sobre el conjunto de validación, lo que provocó un experimento adicional en el modelo 2.0 en donde se incrementó la complejidad del modelo y se redujo el tamaño del conjunto de entrenamiento lo que generó un escenario con sobreajuste real. Gracias a estas condiciones se aplicó una técnica de regularización en donde se mostró mejorar el desempeño del modelo, el mejor resultado obtuvo un alpha de 1 (MSE de validación de 30.348), frente a un MSE de 30.652 con un

alpha de 0.1, lo que confirmó que dicha técnica funcionó de manera correcta.

8 Conclusión

Durante este artículo se abordó el problema de predecir el peso de los perros en adopción a partir de sus características físicas, conductuales y de historial de ingreso, todo esto mediante un proceso que incluyó la limpieza y preparación de los datos (ETL), un análisis exploratorio de las variables disponibles, una reducción de dimensionalidad mediante PCA y finalmente la construcción y evaluación de dos modelos de regresión lineal, uno implementado desde cero mediante descenso de gradiente (modelo 1.0) y el segundo utilizando el framework scikit-learn (modelo 2.0).

Ambos modelos lograron un desempeño prácticamente idéntico, explicando alrededor del 88% de la variabilidad del peso de los perros, con un error absoluto promedio de poco más de 3 kg sobre el conjunto de prueba. Esta coincidencia entre ambas implementaciones resulta particularmente relevante debido a que confirma que el modelo 1.0 aunque no se haya creado con ningún framework, se converge correctamente hacia la misma solución que se obtiene en el modelo 2.0.

El análisis de sesgo y varianza realizado de manera consistente en ambos modelos mediante la comparación contra un modelo trivial y la evaluación sobre conjuntos independientes de entrenamiento, validación y prueba, gracias a eso se mostró que en ambos casos el sesgo era bajo y una varianza baja lo que sitúa a los dos modelos en un punto de buen ajuste sin evidencia de underfitting o overfitting. Esta conclusión se vio reforzada por la verificación del error residual del modelo es consistente con la variación natural que existe dentro de cada categoría de tamaño de los perros, es decir que con el ruido irreducible propio del fenómeno y no con

información que el modelo esté dejando de aprovechar.

Finalmente, la aplicación de regularización L2 esta fue hecha en ambos modelos pero en el modelo 1.0 fue mediante el uso del lambda y en el 2.0 se utilizó Ridge. En este caso se confirmó que en ambos modelos esta técnica no aporta una mejora sobre el desempeño ya alcanzado. De hecho este resultado constituye una conclusión metodológicamente válida debido a que la regularización está diseñada para corregir problemas de varianza alta y al haberse determinado que ninguno de los dos modelos presenta dicho problema era de esperarse que al penalizar los coeficientes no generaba ninguna mejora adicional. Esta conclusión se validó adicionalmente mediante un experimento adicional en el que se aumentó la complejidad del modelo 2.0 (reduciendo el tamaño de conjunto de entrenamiento), gracias a esto se observó una mejora real y auténtica en el desempeño del modelo al aplicar la técnica de regularización. Gracias a esto, podemos confirmar que la técnica funciona correctamente si y sólo si está bajo las condiciones esperadas.

En conjunto, los resultados obtenidos indican que las variables utilizadas particularmente la categoría de tamaño del perro son suficientes para construir un modelo de regresión lineal con una capacidad predictiva sólida y estable y que dicho modelo generaliza de manera confiable a los datos no vistos durante su entrenamiento.

8 Referencias

- [1] <https://www.shelteranimalscount.org/shelter-animals-counts-estimation-model/>
- [2] <https://www.sciencedirect.com/science/article/abs/pii/S0957417422009083>
- [3] https://books.google.com.mx/books/about/Applied_Predictive_Modeling.html?id=xYRDAAAQBAI
- [4] <https://www.statlearning.com/>
- [5] <https://archive.org/details/datawarehouseetl0000kimb>
- [6] https://archive.org/details/exploratorydataa0000tuke_7616
- [7] <https://link.springer.com/book/10.1007/b98835>
- [8] <https://www.questionpro.com/blog/es/regresion-multiple/>
- [9] <https://www.coursera.org/specializations/machine-learning-introduction>